FitBuddy – AI Fitness Plan Generator using Gemini Models

Project Description:

FitBuddy is a web-based application that uses AI to generate personalized workout plans and nutrition tips based on a user's fitness goals, such as weight loss, muscle gain, flexibility, or general wellness. Built with FastAPI and integrated with Google's Gemini AI models, FitBuddy simplifies the process of creating structured and effective fitness routines.

The application takes in user information—like name, user ID, age, weight, goal, and preferred workout intensity—and generates a customized 7-day fitness plan. Users can also submit feedback to update and refine their plan using AI. A complementary nutrition or recovery tip is also provided based on their selected goal.

With a clean user interface and database support via SQLite, FitBuddy ensures each user's data and generated plan are stored and accessible. It offers both REST API endpoints and an interactive HTML interface for ease of use. When no Gemini API key is configured, a clearly labeled local demo generator keeps the complete application working.

Scenarios:

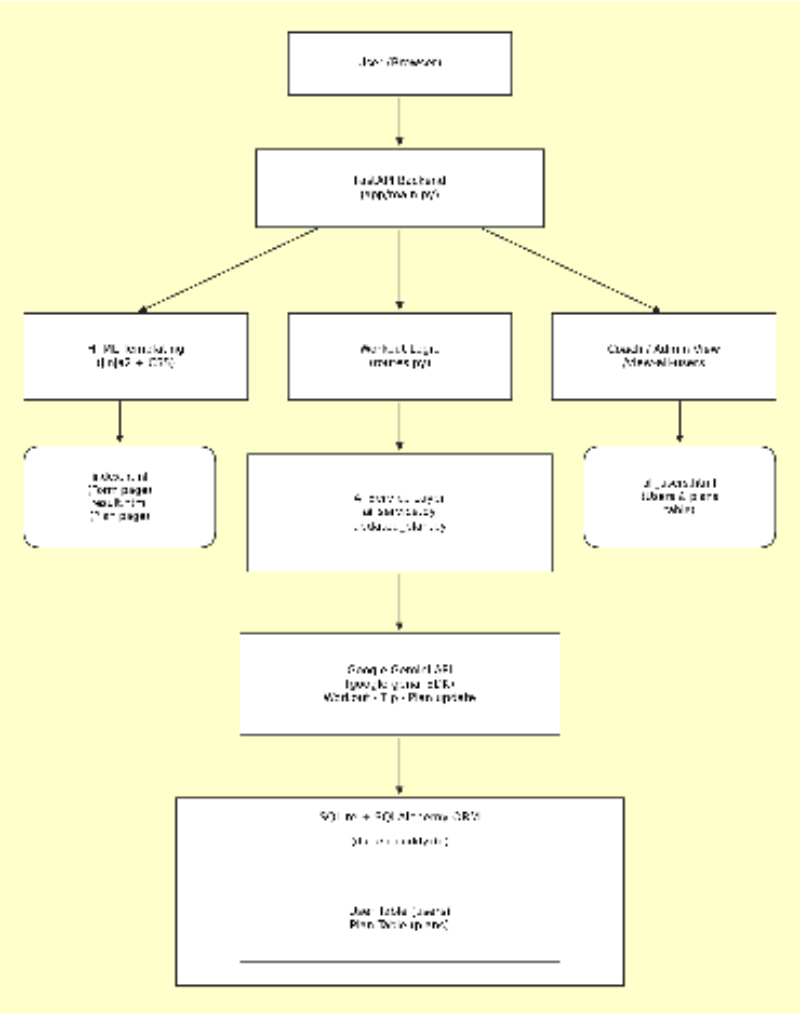
Scenario 1 : A user inputs their personal details including name, user ID, age, weight, fitness goal (e.g., "weight loss" or "muscle gain"), and preferred workout intensity ("high", "medium" or "low") to receive a personalized 7-day workout plan. The system generates a day-by-day schedule tailored to the user's fitness objective and intensity preference. This AI-driven customization enhances user motivation and adherence by providing structured and goal-specific routines.

Scenario 2: A user submits feedback on their existing workout plan, such as suggesting changes like "more focus on cardio" or "include more rest days." The system uses this input to regenerate an updated workout plan, refining it based on the user's evolving needs. This feedback loop ensures that users stay engaged and receive dynamic fitness support as their preferences or progress change.

Scenario 3: A user requests a nutrition or recovery tip based on their selected fitness goal. The platform delivers a concise and relevant suggestion (e.g., "Include protein in your post-workout meal" for muscle gain), which complements the workout plan. These targeted tips help users improve results by aligning their diet and recovery practices with their physical training regimen.

Scenario 4: An admin or coach accesses the platform to view all registered users and their workout plans through a dedicated dashboard. This view allows them to track individual progress, compare original and updated plans, and monitor user activity. It streamlines oversight for trainers managing multiple clients or students.

Technical Architecture:



Pre - requisites:

1. FastAPI Framework Knowledge: FastAPI Documentation
2. Gemini API Familiarity: Google Generative AI Documentation
3. HTML, CSS, and Jinja2 Template Skills: W3Schools HTML/CSS/Jinja2 Tutorials
4. Python Programming Proficiency: Python Official Docs
5. Version Control with Git: Git Documentation
6. SQLAlchemy and SQLite Basics: SQLAlchemy Docs
7. Environment Setup (pip, virtualenv, or conda): Virtualenv Guide
8. Uvicorn ASGI Server: Uvicorn Docs

Project Workflow:

Milestone 1. Model selection and Architecture

- **Activity 1.1. Research and Select the Appropriate Generative AI Model**
- **Activity 1.2. Define the architecture of the application**
- **Activity 1.3. Set up the development Environment**

Milestone 2. Core functionalities Development

- **Activity 2.1. Develop the core functionalities**
- **Activity 2.2. Implement the FastAPI Backend to Manage Routing and User Input Processing**

Milestone 3. routes.py Development

- **Activity 3.1. Writing the Main Application Logic in routes.py**

Milestone 4. Frontend Development

- **Activity 4.1. Designing and Developing User Interface**
- **Activity 4.2. Creating Dynamic Templates with FastAPI's Jinja2**

Milestone 5. Deployment

- **Activity 5.1. Preparing the Application for Local Deployment**
- **Activity 5.2. Testing and Verifying Local Deployment**

Milestone 1. Model selection and Architecture

In this milestone, we focus on selecting the appropriate **generative AI models from Google's Gemini family** for the **FitBuddy** application. This includes evaluating the capabilities of the available options to ensure they align with the application's core functionalities—such as generating personalized workout plans, updating them based on feedback, and providing tailored nutrition or recovery advice.

Activity 1.1: Research and Select the Appropriate Generative AI Model

Understand the Project Requirements

FitBuddy's AI model needs to:

- Generate structured 7-day workout plans
- Provide concise and actionable nutrition tips
- Update workout plans based on user feedback
- Perform well in real-time, web-based interaction

Model Evaluation

Open-source Hugging Face models were considered as alternatives:

- T5 & BART – For text generation (workout/nutrition)
- DistilBERT – For potential intent classification
- Bloom – For general purpose generation

While these models are capable of basic text generation, they have notable limitations. They require:

Hosting with Hugging Face or a local GPU setup

Fine-tuning or complex prompting to achieve structured plans

Final Model Selection

Google Gemini models were selected for the application, accessed through the google-genai SDK. The model names are configurable through environment variables, so a different Gemini model can be used without changing code:

Workout model (GEMINI_WORKOUT_MODEL, default gemini-3-flash)

→ Used for structured 7-day workout generation and plan updating

→ Delivers rich, context-aware, formatted responses reliably

Tip model (GEMINI_TIP_MODEL, default gemini-3-flash)

→ Used for generating fast, practical nutrition or recovery tips

→ Lightweight and efficient for quick API calls

Reasons for Choosing Gemini:

API-based, easy to integrate with FastAPI

Naturally understands structured formatting (days, sections)

High accuracy with minimal prompt tuning

Scalable and fast for feedback-based revisions

Prompts are written conservatively (no diagnosis, extreme dieting, unsafe exercise, or guaranteed results)

Activity 1.2. Define the architecture of the application

Architecture Overview: FitBuddy follows a modular FastAPI-based architecture-

Frontend: HTML + Jinja2 templates and CSS for user input and display

Backend: FastAPI for route handling and logic

AI Layer: Google Gemini API (workout/update model and tip model)

Database: SQLite with SQLAlchemy for persistence

Frontend Responsibilities: Users interact via-

base.html – Shared layout (navigation and footer)

index.html – Input form

result.html – Plan, tip, feedback

all_users.html – Admin panel

error.html – Validation and not-found messages

Backend Responsibilities:

- Receives form data
- Validates input with Pydantic
- Calls Gemini models
- Stores user info and plans
- Renders dynamic templates

AI Integration Points:

- Workout generation & feedback updates → workout model
- Nutrition tips → tip model

Activity 1.3 Set up the development Environment:

Install Python & Pip : Ensure Python and pip are installed.

Create Virtual Environment :

```
python -m venv .venv
.venv\Scripts\activate        # Windows
source .venv/bin/activate     # macOS / Linux
```

Install Required Libraries :

```
pip install -r requirements.txt
```

The requirements.txt file lists the project dependencies:

```
fastapi>=0.115,<1.0
uvicorn[standard]>=0.30,<1.0
jinja2>=3.1,<4.0
sqlalchemy>=2.0,<3.0
python-multipart>=0.0.9,<1.0
pydantic>=2.7,<3.0
pydantic-settings>=2.4,<3.0
python-dotenv>=1.0,<2.0
google-genai>=1.0,<2.0
pytest>=8.0,<9.0
httpx>=0.27,<1.0
```

Run Server :

```
python -m uvicorn app.main:app --reload
```

Visit http://127.0.0.1:8000 for the app and /docs for API testing. Run the command from the project root so that the templates and static folders are found.

Set Up Project Structure :

Create folders: /app (Python files), /templates (HTML), /static (CSS and JavaScript), /tests and /data (SQLite database)

```
FitBuddy/

    requirements.txt          # Project dependencies
    .env.example              # Environment variable template
    README.md

    app/
        __init__.py
        main.py               # FastAPI entry point
        config.py             # Settings loaded from .env
        routes.py             # Core route handlers
        database.py           # SQLAlchemy engine, session, init_db()
        models.py             # User and Plan ORM models
        schemas.py            # Pydantic models for validation
        gemini_generator.py   # Workout plan generator
        gemini_flash_generator.py # Nutrition tip generator
        updated_plan.py       # Feedback-based plan updater
        services/
            ai_service.py     # Gemini calls + local demo fallback

    templates/
        base.html             # Shared layout
        index.html            # User input form
        result.html           # Workout plan, tip, feedback
        all_users.html        # Admin dashboard
        error.html            # Error page

    static/
        css/style.css         # Application styling
        js/app.js             # Small UI helpers

    tests/
        test_app.py           # Automated tests

    data/
        fitbuddy.db           # Local SQLite database (auto-generated)
```

Milestone 2. Core functionalities Development

Activity 2.1 Develop the core functionalities:

Application Configuration

File: config.py

Application settings are defined with pydantic-settings and loaded from a .env file. This keeps the Gemini API key and model names outside the source code.

```
class Settings(BaseSettings):
    app_name: str = "FitBuddy"
    database_url: str = "sqlite:///./data/fitbuddy.db"
    gemini_api_key: str = ""
    gemini_workout_model: str = "gemini-3-flash"
    gemini_tip_model: str = "gemini-3-flash"

    model_config = SettingsConfigDict(env_file=".env", extra="ignore")
```

Workout Plan Generation

Function: generate_workout_gemini()
File: gemini_generator.py (logic in services/ai_service.py)

Implements the core logic to generate a personalized 7-day workout plan using user inputs such as name, age, weight, goal (e.g., muscle gain, weight loss) and intensity (low, medium, high). The prompt is sent to the configured Gemini workout model, which returns a structured, day-wise plan including:

- Focus of the day
- Warm-up
- Main workout (exercise names, sets/reps or duration)
- Rest / recovery guidance

The plan always contains exactly seven labeled days with at least one recovery day, and ends with a short safety note. The output is returned to routes.py and rendered on the result.html page using <pre> blocks for formatting.

```
def generate_workout(name, age, weight, goal, intensity):
    prompt = f"""
You are FitBuddy, a conservative wellness-planning assistant.
Create a practical 7-day general fitness plan for:
Name: {name}
Age: {age}
Weight: {weight} kg
Goal: {goal}
Preferred intensity: {intensity}

Return plain text with exactly seven labeled days. For every day include Focus, Warm-up,
Main workout with exercise names and sets/reps or duration, and Rest/recovery guidance.
Include at least one recovery/rest day. Avoid diagnosis, extreme calorie restriction,
```

```
unsafe exercise, or guaranteed results. End with a short safety note.
"""
    try:
        return _generate(prompt, settings.gemini_workout_model) or demo_workout(...)
    except Exception:
        return demo_workout(name, age, weight, goal, intensity)
```

Nutrition Tip Generation

Function: generate_nutrition_tip_with_flash()
File: gemini_flash_generator.py (logic in services/ai_service.py)

This function generates a concise, practical nutrition or recovery tip tailored to the user's selected goal. It uses the lightweight tip model to deliver a fast, helpful response such as hydration advice, protein suggestions, or general recovery best practices. The tip is limited to 2–4 sentences and is displayed beneath the workout plan in result.html.

```
def generate_tip(goal):
    prompt = f"""
Give one concise nutrition or recovery tip for the fitness goal "{goal}".
Use general wellness advice only. Avoid calorie prescriptions, diagnosis, medication advice,
extreme restriction, and guaranteed outcomes. Keep it to 2–4 sentences.
"""
    try:
        return _generate(prompt, settings.gemini_tip_model) or demo_tip(goal)
    except Exception:
        return demo_tip(goal)
```

Feedback-Based Plan Updating

Function: update_workout_plan()
File: updated_plan.py

This function enables the user to revise their original plan by submitting feedback (e.g., "Add yoga", "Include more cardio"). The original plan, the feedback, and the user's goal and intensity are passed to Gemini, which returns a modified 7-day plan with the suggested updates while keeping at least one recovery day. The updated plan is then stored and displayed alongside the nutrition tip.

```
def update_workout_plan(original_plan, feedback, goal, intensity):
    if not settings.gemini_api_key.strip():
        return _demo_update(original_plan, feedback)
    prompt = f"""
You are updating an existing FitBuddy wellness plan.
Goal: {goal}
Intensity: {intensity}
Original plan:
{original_plan}
User feedback:
{feedback}
Return a revised 7-day plan. Apply reasonable feedback, keep at least one recovery/rest day,
avoid unsafe exercise or extreme dieting, and finish with a brief safety note.
"""
    try:
        client = genai.Client(api_key=settings.gemini_api_key)
        response = client.models.generate_content(
            model=settings.gemini_workout_model, contents=prompt,
```

```
            config=types.GenerateContentConfig(temperature=0.7, max_output_tokens=3000))
        return (response.text or "").strip() or _demo_update(original_plan, feedback)
    except Exception:
        return _demo_update(original_plan, feedback)
```

Local Demo Fallback

Functions: demo_workout(), demo_tip(), _demo_update()

If no Gemini API key is configured, or if a Gemini call fails, the application automatically falls back to a local demo generator. The demo plan is clearly labeled ("FitBuddy demo plan for …"), so the complete application can be run and tested without network access or an API key.

User & Plan Storage

Files: database.py, models.py

All user information (ID, name, age, weight, goal, intensity) and AI-generated plans are stored in a SQLite database via SQLAlchemy ORM. Feedback-based updates are saved in a separate updated_plan field to preserve both versions (original and updated). The tables are created automatically at startup by init_db().

Table	Columns
users	id (PK), user_id (unique), name, age, weight, goal, intensity, created_at
plans	id (PK), user_id (unique), original_plan, updated_plan, nutrition_tip, feedback, created_at, updated_at

```
class User(Base):
    __tablename__ = "users"
    id: Mapped[int] = mapped_column(Integer, primary_key=True)
    user_id: Mapped[str] = mapped_column(String(80), unique=True, index=True)
    name: Mapped[str] = mapped_column(String(120))
    age: Mapped[int] = mapped_column(Integer)
    weight: Mapped[float] = mapped_column(Float)
    goal: Mapped[str] = mapped_column(String(80))
    intensity: Mapped[str] = mapped_column(String(20))

class Plan(Base):
    __tablename__ = "plans"
    id: Mapped[int] = mapped_column(Integer, primary_key=True)
    user_id: Mapped[str] = mapped_column(String(80), unique=True, index=True)
    original_plan: Mapped[str] = mapped_column(Text)
    updated_plan: Mapped[str | None] = mapped_column(Text, nullable=True)
    nutrition_tip: Mapped[str] = mapped_column(Text)
    feedback: Mapped[str | None] = mapped_column(Text, nullable=True)
```

Storing a plan is handled by save_plan() in routes.py. Generating a plan again for an existing User ID replaces the original plan and clears any earlier update and feedback:

```
def save_plan(db, payload, workout, tip):
    plan = db.scalar(select(Plan).where(Plan.user_id == payload.user_id))
    if plan is None:
        plan = Plan(user_id=payload.user_id, original_plan=workout, nutrition_tip=tip)
        db.add(plan)
```

```
    else:
        plan.original_plan = workout
        plan.updated_plan = None
        plan.feedback = None
        plan.nutrition_tip = tip
    return plan
```

Admin View of All Users

Route: /view-all-users Template: all_users.html

Displays a list of all users and their respective plans. Rendered with Jinja2 using a {% for user in users %} loop, it allows administrators to see original and updated plans clearly in a structured table format. Each row also has a Delete button to remove a user and their saved plans.

```
@router.get("/view-all-users", response_class=HTMLResponse)
def view_all_users(request: Request, db: Session = Depends(get_db)):
    users = db.scalars(select(User).order_by(User.created_at.desc())).all()
    plans = {p.user_id: p for p in db.scalars(select(Plan)).all()}
    return templates.TemplateResponse(request=request, name="all_users.html",
        context={"title": "All Users", "users": [row(u, plans.get(u.user_id)) for u in users]})
```

Activity 2.2: Implement the FastAPI Backend to Manage Routing and User Input Processing

Define Routes in FastAPI

All routing logic is handled in routes.py. Each route is linked to the corresponding function developed in Activity 2.1. The router is registered in main.py, which also mounts the /static folder and creates the database tables at startup.

```
@asynccontextmanager
async def lifespan(app: FastAPI):
    init_db()
    yield

app = FastAPI(title=settings.app_name, version="1.0.0", lifespan=lifespan)
app.mount("/static", StaticFiles(directory="static"), name="static")
app.include_router(router)
```

Process User Input

HTML Form: Created in index.html

Users enter: name, user_id, age, weight, goal, and intensity

FastAPI captures inputs using Form(...) parameters

Pydantic schemas (UserInput, FeedbackRequest) are used to validate structure

Field	Validation rule (schemas.py)
name	Text, 2–120 characters
user_id	Text, 2–80 characters; letters, digits, underscore and hyphen only
age	Integer, 13–100

weight	Number greater than 0, up to 500 (kg)
intensity	One of: low, medium, high
feedback	Text, 3–2000 characters

Integrate Gemini API Calls

Each route handler connects directly to the respective Gemini-powered function:

- /generate-workout →
  - generate_workout_gemini() for workout plan
  - generate_nutrition_tip_with_flash() for tip
- /submit-feedback →
  - Retrieves plan
  - Sends original + feedback to update_workout_plan()

All AI-generated responses are returned cleanly and injected into the frontend templates for display.

Milestone 3. routes.py Development

Milestone 3 focuses on building the routing logic inside the routes.py file, which acts as the core operational layer of the FitBuddy application. It bridges the frontend templates, AI generation modules (workout and tip models), and the SQLite database. Each route defines how user inputs are collected, processed, and responded to — creating a dynamic, personalized experience for fitness planning and feedback-based updates.

Activity 3.1: Writing the Main Application Logic in routes.py

Defining Core routes in routes.py:

Separate routes were implemented to serve each of FitBuddy's main functionalities:

1. / : Displays the user form via index.html
2. /generate-workout: Processes user input and generates a personalized workout plan
3. /submit-feedback: Updates the workout plan based on user feedback
4. /view-all-users: Displays an admin dashboard showing all users and their plans

Set up Route Handlers for Each Feature

/ – Home Route

Returns the homepage (index.html)

No form processing logic

/generate-workout – Plan Generator

Receives user input using FastAPI's Form(...)

Constructs a UserInput Pydantic model (invalid input renders error.html)

Calls:

generate_workout_gemini(...) → Generates workout plan via Gemini

generate_nutrition_tip_with_flash(...) → Gets nutrition tip via Gemini

Stores:

User details (created, or updated if the User ID exists)

Plan with save_plan(...)

Returns result.html with dynamic data

/submit-feedback – Update Plan with Feedback

Captures feedback and user_id

Retrieves the user and original plan from DB (renders error.html with 404 if not found)

Uses update_workout_plan(...) to revise the plan via Gemini

Saves the revised plan and the feedback text in the plans table

Renders updated result on result.html

/view-all-users – Admin View

Queries all users and their plans from the database

Passes them to all_users.html

Admin can view:

Name, User ID, Age, Weight, Goal, Intensity

Original and updated workout plans

A delete option (/delete-user/{user_id})

Integrate Gemini AI Responses in Each Function

Each AI-related route calls Google's Gemini models with specific prompts:

Workout Plan Generation:

generate_workout_gemini() sends name, age, weight, fitness goal and intensity to Gemini to return a structured 7-day workout plan.

Nutrition Tip Generation:

generate_nutrition_tip_with_flash() generates a practical tip related to the user's fitness goal.

Workout Plan Update via Feedback:

update_workout_plan() passes the original plan + user feedback to Gemini and returns a revised plan.

All responses are returned to the user through result.html, maintaining clarity using <pre> blocks for formatting.

Workout generation route

```
@router.post("/generate-workout", response_class=HTMLResponse)
def generate_workout_form(request: Request, name: str = Form(...), user_id: str = Form(...),
                          age: int = Form(...), weight: float = Form(...),
                          goal: str = Form(...), intensity: str = Form(...),
                          db: Session = Depends(get_db)):
    payload = UserInput(name=name.strip(), user_id=user_id.strip(), age=age,
                        weight=weight, goal=goal, intensity=intensity)
    user = db.scalar(select(User).where(User.user_id == payload.user_id))
    if user is None:
        user = User(**payload.model_dump())
        db.add(user)
    else:
        for key, value in payload.model_dump().items():
            setattr(user, key, value)
    workout = generate_workout_gemini(**payload.model_dump())
    tip = generate_nutrition_tip_with_flash(payload.goal)
    plan = save_plan(db, payload, workout, tip)
    db.commit()
    return templates.TemplateResponse(request=request, name="result.html",
        context={"title": "Your FitBuddy Plan", "user": user, "plan": plan,
                 "display_plan": plan.original_plan, "is_updated": False})
```

Submit Feedback and update plan route

```
@router.post("/submit-feedback", response_class=HTMLResponse)
def submit_feedback_form(request: Request, user_id: str = Form(...), feedback: str = Form(...),
                         db: Session = Depends(get_db)):
    user = db.scalar(select(User).where(User.user_id == user_id))
    plan = db.scalar(select(Plan).where(Plan.user_id == user_id))
    if user is None or plan is None:
        return templates.TemplateResponse(request=request, name="error.html",
            context={"title": "User not found",
                     "message": "Generate a plan for this User ID first."}, status_code=404)
    feedback_model = FeedbackRequest(feedback=feedback.strip())
    revised = update_workout_plan(plan.original_plan, feedback_model.feedback,
                                  user.goal, user.intensity)
    plan.updated_plan = revised
    plan.feedback = feedback_model.feedback
    db.commit()
    return templates.TemplateResponse(request=request, name="result.html",
        context={"title": "Updated FitBuddy Plan", "user": user, "plan": plan,
                 "display_plan": revised, "is_updated": True})
```

REST API routes

In addition to the HTML pages, FitBuddy exposes JSON endpoints (documented interactively at /docs):

Method	Route	Purpose
GET	/	Home page with the input form
GET	/health, /api/health	Service health check
POST	/generate-workout	Form submission: generate plan + tip, render result.html
POST	/submit-feedback	Form submission: revise plan from feedback, render result.html
GET	/view-all-users	Admin dashboard (all_users.html)
POST	/delete-user/{user_id}	Delete a user and plan, redirect to dashboard
POST	/api/plans	JSON: generate and save a plan
POST	/api/plans/{user_id}/feedback	JSON: update a plan with feedback
GET	/api/users	JSON: list all users with original/updated plans
DELETE	/api/users/{user_id}	JSON: delete a user

Example request body for POST /api/plans:

```
{
  "name": "Demo User",
  "user_id": "demo001",
  "age": 20,
  "weight": 60,
  "goal": "general wellness",
  "intensity": "medium"
}
```

Milestone 4. Frontend Development

Milestone 4 focuses on developing a user-friendly and visually structured interface for FitBuddy, the AI-powered workout generator. This involves building responsive and interactive web pages using HTML, CSS, and Jinja2 templating. Each template is dynamically rendered through FastAPI, ensuring users can easily submit input, receive personalized plans, provide feedback, and allow administrators to manage data effectively. This milestone guarantees an engaging and intuitive fitness planning experience for all users.

Activity 4.1 Designing and Developing the User Interface

Set Up the Base HTML Structure

Developed index.html as the main entry point for user input.

Structured the form to capture: name, user_id, age, weight, goal, and intensity.

Each field is properly labeled and grouped using semantic HTML, with built-in browser validation (required, min/max, pattern).

A shared base.html layout provides the top navigation bar (Generate, All Users) and footer, and all other pages extend it.

The design uses a dark navy theme with mint-green accents and bold typography to create a clean, fitness-oriented interface.

Navigation is kept minimal for focus, and user flow directs clearly from input to output.

Design a Responsive Layout Using CSS

A single stylesheet, static/css/style.css, is shared by all pages.

Layouts use **Flexbox** and **CSS Grid** to arrange the hero section, profile cards and forms.

Media queries (at 850px and 560px) ensure mobile responsiveness.

Buttons have consistent styling for primary, secondary and danger actions.

Input fields and result blocks have adequate padding, borders, shadows, and rounded corners for modern aesthetics.

Overall color scheme supports readability on a dark background.

A small script (static/js/app.js) disables a form's submit button while the AI request is running and asks for confirmation before a user is deleted.

Create Separate Pages for Each Core Functionality

Jinja2-powered HTML templates were created under /templates:

base.html: Shared layout that the other templates extend.

index.html: A user input form with fields for fitness goal, intensity, and basic personal information.

result.html: Displays the profile summary, the 7-day AI generated workout plan, a

nutrition tip, and accepts user feedback.

all_users.html: Admin panel to view and delete users and review both original and updated plans.

error.html: Friendly message for invalid input or an unknown User ID.

Templates are modular, and UI transitions smoothly between steps.

Forms are submitted to specific FastAPI endpoints via POST methods

Activity 4.2 Creating Dynamic Templates with FastAPI's Jinja2

Integrate Jinja2 Templating for Dynamic Content Rendering

Used FastAPI's Jinja2Templates for template rendering:

```
from fastapi.templating import Jinja2Templates
templates = Jinja2Templates(directory="templates")
```

Each route uses TemplateResponse to render the proper HTML with passed context:

```
return templates.TemplateResponse(request=request, name="result.html",
    context={"title": "Your FitBuddy Plan",
            "user": user,                # name, age, weight, goal, intensity
            "plan": plan,                # nutrition_tip, feedback
            "display_plan": plan.original_plan,
            "is_updated": False})
```

This allows dynamic plan and nutrition tip display after form submission.

Bind Backend Data to HTML Templates

index.html

Form elements use name="..." to bind to backend Form(...) parameters.

Submits data to /generate-workout.

result.html

Uses:

{{ user.name }}, {{ user.goal }}, {{ user.intensity }}, etc. to show user data.

{{ display_plan }} and {{ plan.nutrition_tip }} to show AI-generated results.

Shows an "UPDATED PLAN" badge and the last feedback when is_updated is true.

Includes a feedback form submitted via POST to /submit-feedback.

all_users.html

Loops over all user records using:

```
{% for user in users %}
<tr>
  <td><strong>{{ user.name }}</strong><br><small>{{ user.user_id }}</small></td>
  <td>{{ user.age }} yrs<br>{{ user.weight }} kg</td>
  <td>{{ user.goal|title }}<br>{{ user.intensity|title }}</td>
  <td><details><summary>View</summary><pre class="mini">{{ user.original_plan }}</pre></details></td>
  <td>{% if user.updated_plan %}<details><summary>View</summary>
     <pre class="mini">{{ user.updated_plan }}</pre></details>{% else %}—{% endif %}</td>
  <td><form action="/delete-user/{{ user.user_id }}" method="post"
          onsubmit="return confirmDelete()"><button class="danger">Delete</button></form></td>
</tr>
{% endfor %}
```

Milestone 5. Deployment

Now, the focus is on deploying the **FitBuddy** application on a local system using **FastAPI** and **Uvicorn**. The app uses **HTML templates rendered with Jinja2** , alongside AI services powered by **Gemini**. This milestone ensures all environment dependencies are correctly installed, and the app runs smoothly on a local server, preparing it for future deployment on cloud platforms.

Activity 5.1: Preparing the Application for Local Deployment

Set up a virtual environment

Create a virtual environment to manage dependencies and isolate the FitBuddy project from other environments.

Activate the environment and install all necessary dependencies listed in the requirements.txt file, including FastAPI, Uvicorn, SQLAlchemy, and the Google Gen AI SDK.

```
python -m venv .venv
.venv\Scripts\activate        # macOS/Linux: source .venv/bin/activate
python -m pip install --upgrade pip
pip install -r requirements.txt
copy .env.example .env        # macOS/Linux: cp .env.example .env
```

Configure environment variables

Use environment variables to store sensitive data such as the Gemini API Key.

These variables are set in a .env file which is loaded into the application by pydantic-settings (python-dotenv is also included).

Never commit the .env file or share a real API key; .gitignore excludes .env.

```
GEMINI_API_KEY=your_gemini_api_key_here
GEMINI_WORKOUT_MODEL=gemini-3-flash
GEMINI_TIP_MODEL=gemini-3-flash
APP_NAME=FitBuddy
DATABASE_URL=sqlite:///./data/fitbuddy.db
```

If GEMINI_API_KEY is left blank, the application still works using the local demo generator.

Frontend-Backend Rendering via Jinja2

All user interactions are facilitated via **Jinja2**-rendered templates:

index.html: Collects user inputs

result.html: Displays the generated AI plan

all_users.html: Lists all user records for admin view

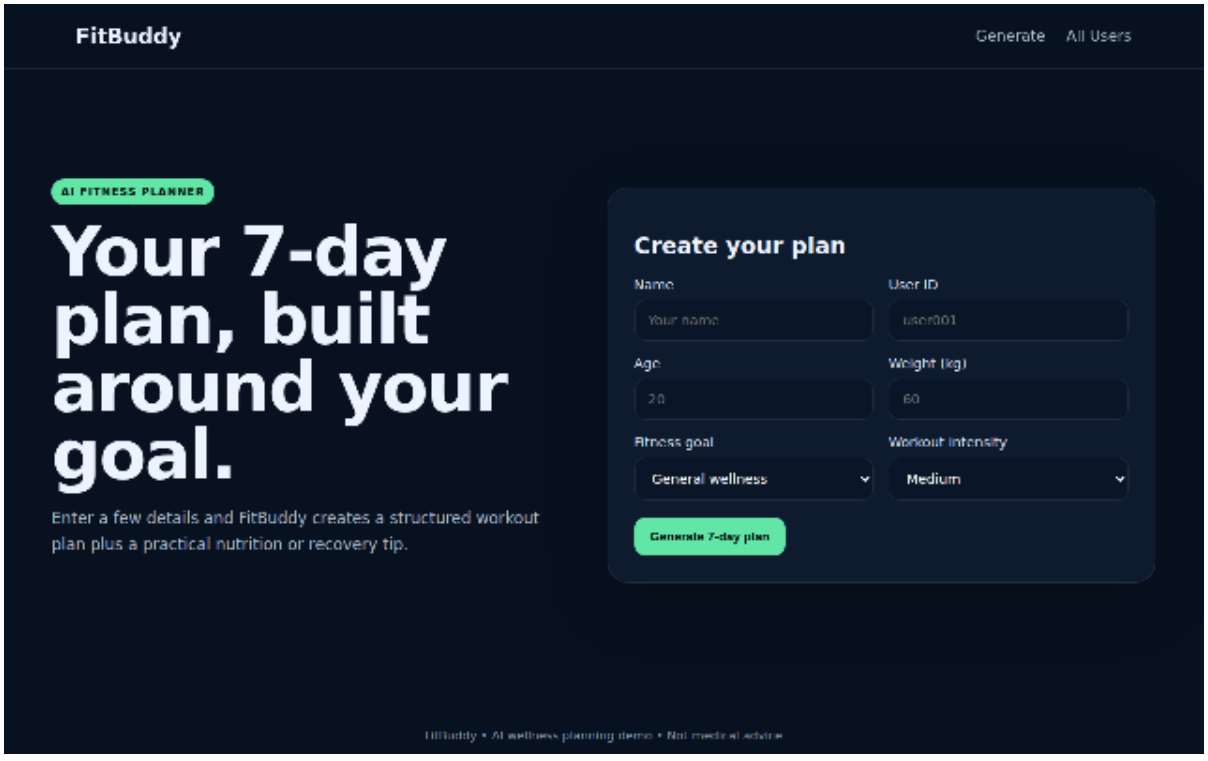
Activity 5.2: Testing and Verifying Local Deployment

Access the Application Locally

The FastAPI development server must be started from the project root using the command (**python -m uvicorn app.main:app –reload**). Once the server is running, the application can be accessed through any web browser by navigating to http://127.0.0.1:8000. This URL loads the main user interface where individuals can input their details and generate personalized fitness plans. Additionally, the interactive API documentation provided by FastAPI can be viewed at http://127.0.0.1:8000/docs, allowing developers to explore and test available endpoints.

Exploring Website's web pages:

Home page:



Description: The homepage of **FitBuddy – AI Workout Generator** features a clean and engaging user interface designed to provide a seamless experience for users seeking personalized

fitness plans. The form prominently displayed on the right allows users to enter essential information including:

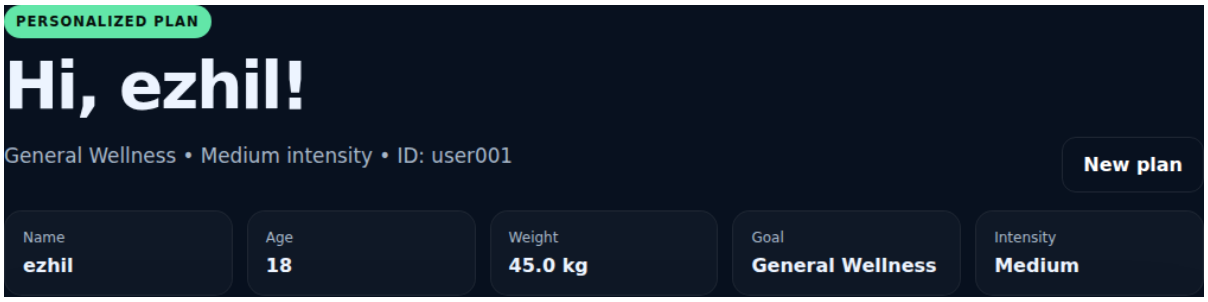
- **Name**
- **User ID**
- **Age**
- **Weight (kg)**
- **Fitness Goal** (Weight loss, Muscle gain, General wellness, Flexibility)
- **Workout Intensity** (Low, Medium, High)

Once users submit this data, they can click the **"Generate 7-day plan"** button, which triggers the AI-powered backend to generate a customized workout and nutrition plan tailored to their goals and fitness profile. The bold headline and dark, high-contrast design reinforce the fitness-centric theme of the application, motivating users to take charge of their health journey.

This intuitive design ensures users of all experience levels can quickly get started with minimal effort.

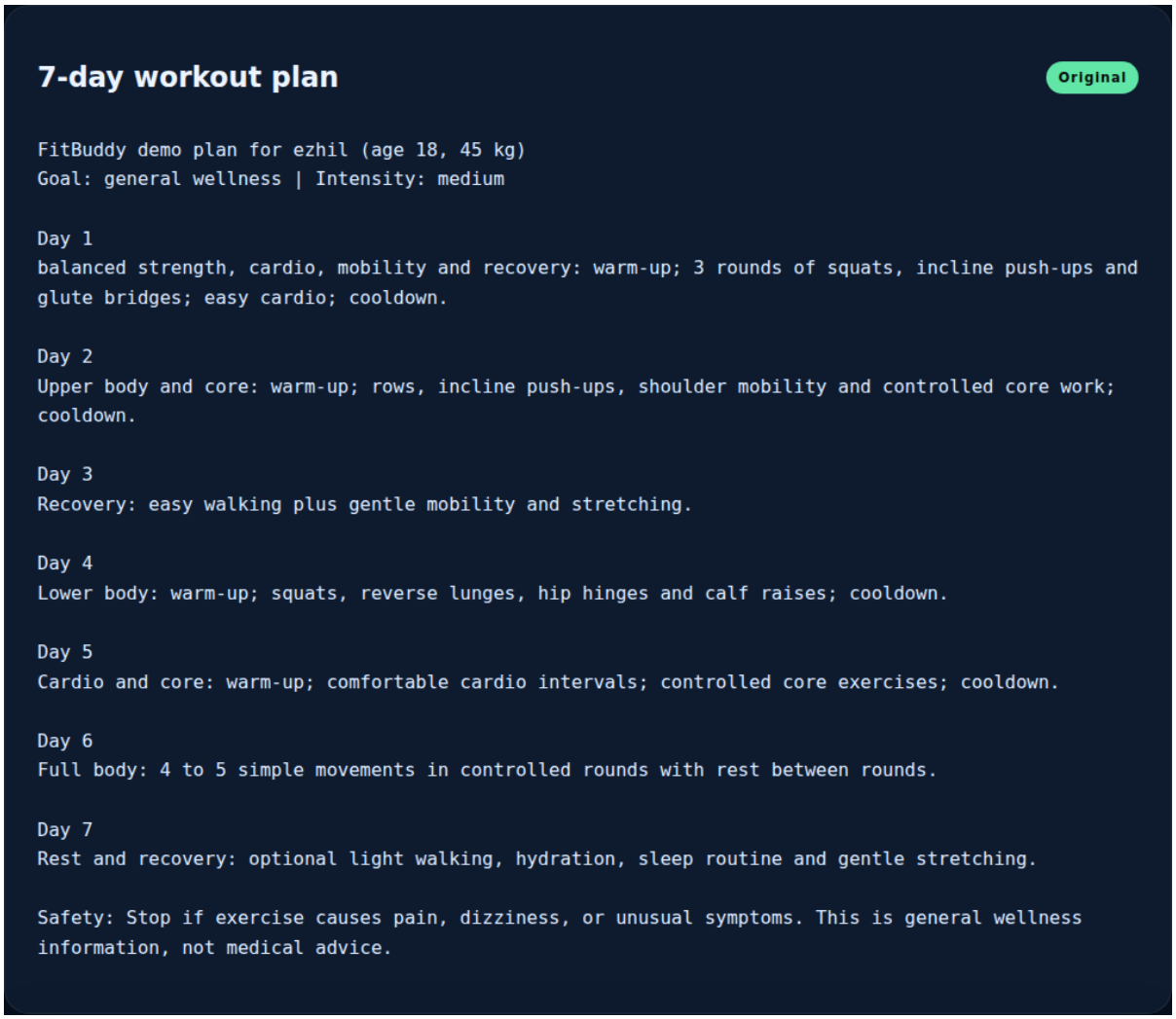
Personalized workout generation page:

User Information:



Description: Once the user submits their details, the application navigates to a personalized workout plan page. This section displays a summary of the user's input including their **Name, Age, Weight, Fitness Goal,** and **Selected Intensity** , along with the User ID needed later for feedback. This personalized display ensures users feel acknowledged and guides the AI engine to generate relevant workout recommendations tailored specifically to their fitness journey.

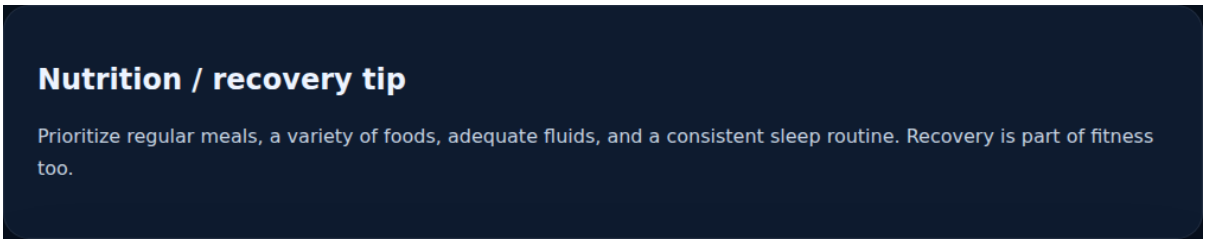
Workout Plan:



Description: FitBuddy generates a personalized, day-wise workout plan tailored to the user's fitness goal and preferred intensity. The workout section on the result page outlines each of the seven days with a clear focus, such as strength, upper or lower body, cardio, core, full body or recovery. For every workout day, a list of exercises is provided along with sets and repetitions (or duration). The plan includes at least one rest and recovery day, basic warm-up and cool-down guidance, and closes with a short safety note. Overall, this section aims to deliver a goal-oriented, easy-to-follow workout schedule that promotes consistency and progress.

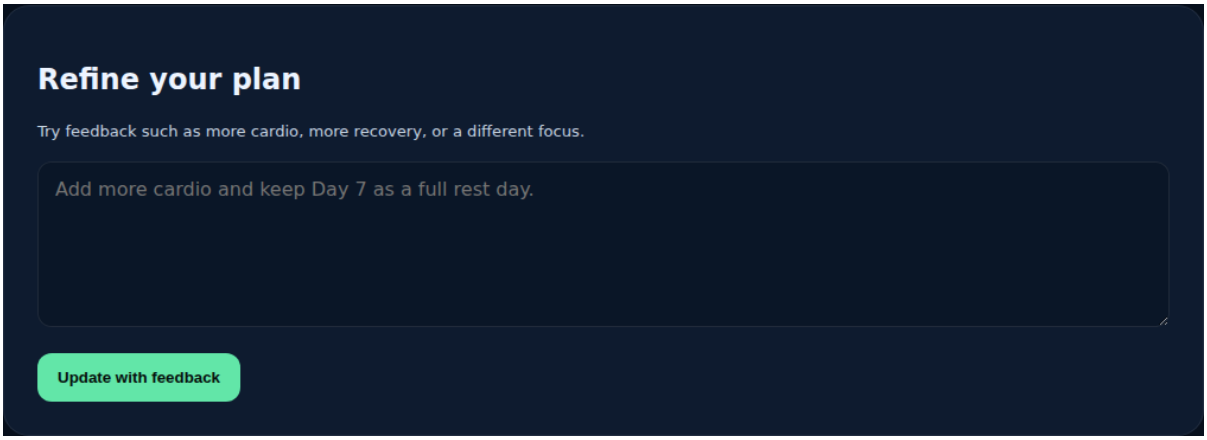
The plan shown above was produced by the built-in local demo generator; when a Gemini API key is configured, the same section displays the richer, model-generated plan.

Nutrition Tip:



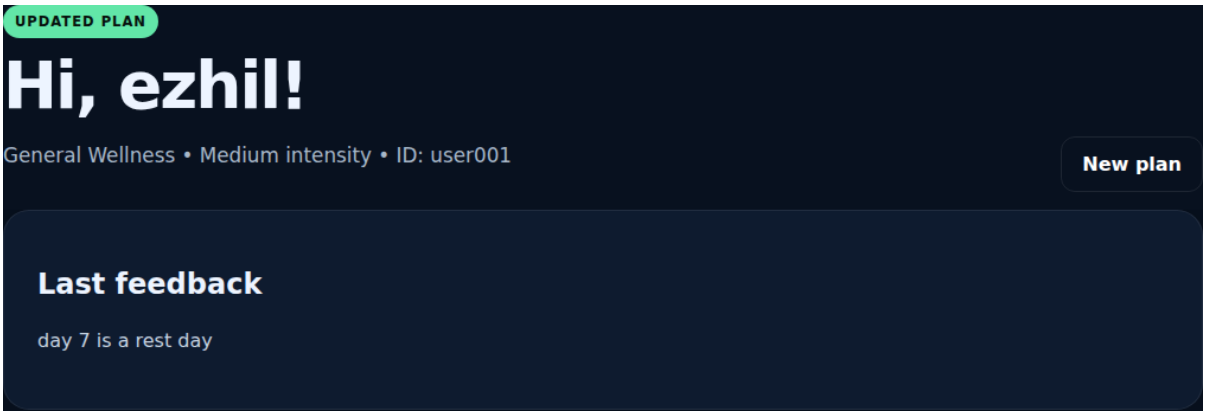
Description: The **Nutrition / recovery tip** section in FitBuddy provides users with concise, practical dietary or recovery advice to complement their workout plans. The tip is tailored to the selected goal and kept to a few sentences; for example, it encourages regular meals, a variety of foods, adequate fluids and a consistent sleep routine. This section is designed to reinforce healthy habits that align with users' fitness goals.

Feedback Page:

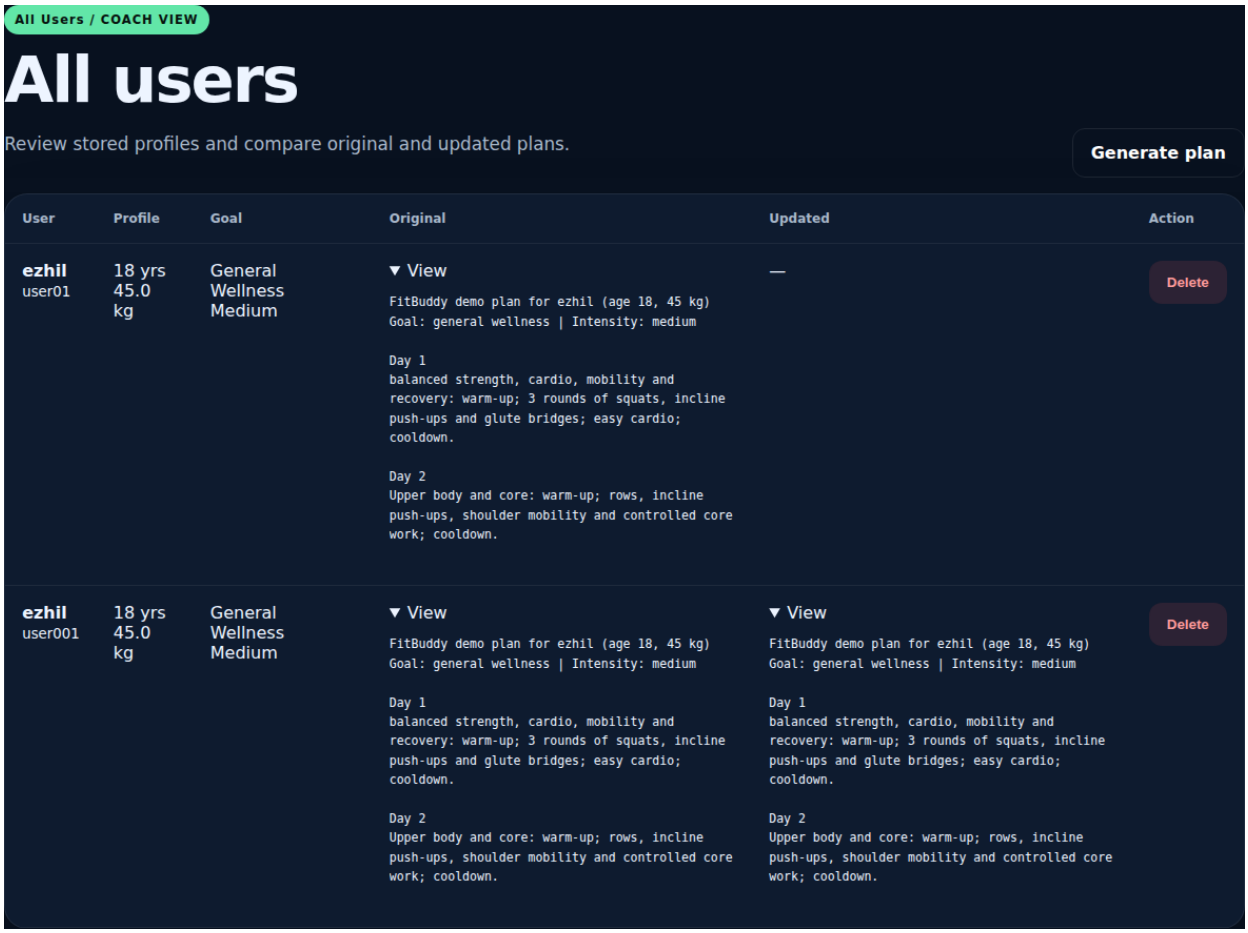


Description: The **"Refine your plan"** form in FitBuddy allows users to personalize and refine their fitness journey by submitting feedback on their current workout plan. The User ID is carried along automatically in a hidden field, so the user only needs to describe what to change —for example more cardio, more recovery, or a different focus. Once submitted, this feedback is processed by the system to intelligently update and adjust the workout plan to better align with the user's goals, comfort, and progress. This dynamic feedback loop ensures that each fitness plan remains adaptive and tailored to the individual's evolving needs.

Once the plan is updated, the page is shown again with an **"UPDATED PLAN"** badge, the plan status changes to **"AI revised"** , and the submitted feedback is displayed under **"Last feedback"** .



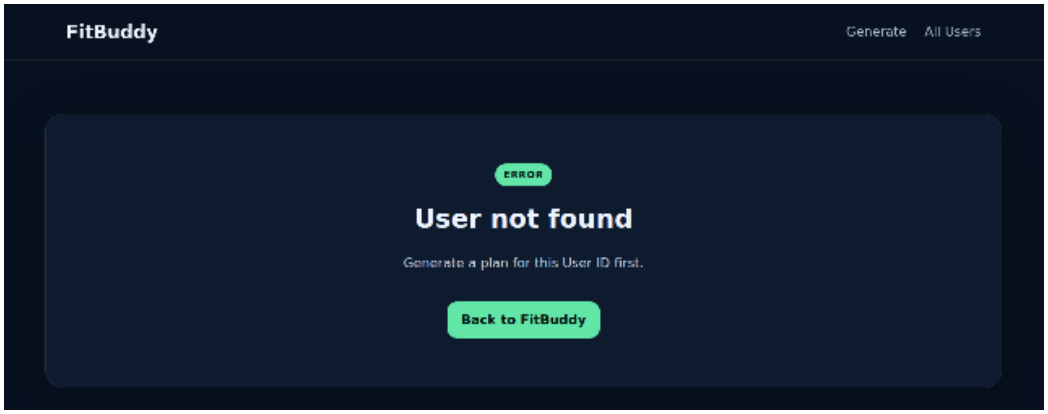
View all users page:



Description: The All Users page in FitBuddy provides an overview of the entire user database, displaying detailed information for each registered user. This includes their personal details such as name, User ID, age, weight, and fitness goal and intensity, along with their original workout plan generated at registration and the updated workout plan if any changes were made based on their submitted feedback. Each plan can be expanded or collapsed, and a Delete button (with a confirmation prompt) removes a user and their plans. This feature enables a

transparent view of how each user's plan has evolved over time, showcasing the adaptability of the FitBuddy system and helping administrators or coaches monitor user progress and engagement effectively.

Error handling:



Description: If the submitted values fail validation, or if feedback is submitted for a User ID that has no plan yet, FitBuddy shows a friendly error page instead of a raw error and provides a button to return to the home page.

Automated Testing

The project includes automated tests (tests/test_app.py) that use FastAPI's TestClient with the demo generator and a separate test database. They can be run from the project root with:

```
pytest -q
```

Test	What it verifies
test_home	Home page loads (status 200) and contains "FitBuddy"
test_health	/api/health returns status "ok"
test_generate	POST /api/plans returns a plan that contains "Day 1"
test_feedback	POST /api/plans/{user_id}/feedback returns an updated plan
test_users	GET /api/users lists the created user
test_delete	DELETE /api/users/{user_id} removes the user

Conclusion:

The FitBuddy – AI Fitness Plan Generator is a comprehensive, user-centric web application that showcases the powerful synergy between artificial intelligence and health-tech solutions. Designed to assist individuals in achieving their fitness goals, FitBuddy uses Google's Gemini models to generate personalized 7-day workout plans and nutrition or recovery tips based on user inputs such as age, weight, fitness goal, and preferred workout intensity.

The application is built using the FastAPI framework for backend logic and API handling, SQLAlchemy with SQLite for persistent data storage, and Jinja2 templating for a clean and responsive frontend. Upon submitting their details on the homepage, users receive a tailored workout plan that includes warm-ups, main workouts, recovery guidance, and a goal-aligned tip. What sets FitBuddy apart is its dynamic feedback feature—users can provide input on the effectiveness or difficulty of their current plan, and the system responds by intelligently updating the plan using generative AI, ensuring ongoing customization and engagement.

Additionally, the application includes an administrative view where all registered users and their respective plans (original and updated) are displayed in a structured format. This functionality supports transparency, monitoring, and further optimization, making FitBuddy suitable for both individual users and coaches or institutions managing multiple fitness profiles. A local demo fallback and an automated test suite make the project easy to run and verify without external dependencies.

Overall, FitBuddy stands as a scalable and adaptable AI-powered solution that not only personalizes the fitness journey but also evolves with the user's progress and preferences. It demonstrates how cutting-edge technology can be integrated into everyday wellness routines to promote healthy, consistent, and goal-oriented lifestyles.